

Intermediate (.env by hand) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/env_config_practice.py`, under its `# Intermediate` section — the same file also holds the [Edge cases exercise](#), under an `# Edge cases` section. Run it with `cd practice && python env_config_practice.py`.

Used later by: the [Build Task](#)'s `load_config()` in `practice/build_task/config.py` — it ships the short `load_dotenv() + os.getenv()` version you land on here, re-written (not imported) so it can return a typed `Config` object.

All examples below assume a `.env` file next to your script — `practice/.env` — containing:

```
# practice/.env
# credentials for the API client
OPENAI_API_KEY="sk-test123"
LOG_LEVEL=DEBUG
```

Basic Version

Approach 1 — the direct way

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path):
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if line == "" or "=" not in line:
            continue
        parts = line.split("=", 1)
        env_vars[parts[0]] = parts[1]
    return env_vars

values = parse_env_by_hand(".env")
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])
```

Expected output:

```
"sk-test123"  
DEBUG
```

Notice `OPENAI_API_KEY` still has its quote marks in the output — this version doesn't know to remove them, it just splits on `=` and keeps whatever's on each side. The comment line (`# credentials...`) is skipped safely too, but only because it happens to contain no `=`, not because this version recognizes it as a comment on purpose.

This version works correctly for what the exercise asks. It's missing type hints, and it builds the dictionary with `parts[0]` / `parts[1]` instead of unpacking directly — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints and tuple unpacking

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path: str) -> dict[str, str]:
    env_vars: dict[str, str] = {}
    for line in open(path):
        line = line.strip()
        if not line or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env_vars[key] = value
    return env_vars

values = parse_env_by_hand(".env")
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])
```

Expected output:

```
"sk-test123"
DEBUG
```

Approach 2 — a docstring, and using `with` to open the file properly

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path: str) -> dict[str, str]:
    """Read a simple KEY=value file and return it as a dict."""
    env_vars: dict[str, str] = {}
    with open(path) as env_file:
        for line in env_file:
            line = line.strip()
            if not line or "=" not in line:
                continue
            key, value = line.split("=", 1)
            env_vars[key] = value
```

```
return env_vars

values = parse_env_by_hand(".env")
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])
```

Expected output:

```
"sk-test123"
DEBUG
```

Difference from Basic: both Intermediate approaches add full type hints (`path: str` , `-> dict[str, str]`) and unpack `key, value = line.split("=", 1)` directly instead of indexing into `parts[0]` / `parts[1]` . Approach 2 also switches to `with open(path) as env_file:` , which closes the file explicitly and correctly even if something inside the loop raises an exception — `for line in open(path):` (Approach 1, and the Basic Version) relies on the file being closed automatically once the loop finishes, which works but is less explicit. Neither approach yet handles the comment line or the quotes on purpose — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — hardened for comments and quotes

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path: str) -> dict[str, str]:
    env_vars: dict[str, str] = {}
    for line in open(path):
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        value = value.strip().strip('"').strip("'")
        env_vars[key] = value
    return env_vars

values = parse_env_by_hand(".env")
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])
```

Expected output:

```
sk-test123
DEBUG
```

This time `OPENAI_API_KEY` prints without quote marks — `value.strip('"').strip("'")` removes a leading/trailing double quote, then a leading/trailing single quote. The comment line is now skipped on purpose (`line.startswith("#")`), not just by accident.

Approach 2 — same parser, plus required-key validation

```
# practice/env_config_practice.py – Intermediate section
class MissingEnvKeyError(Exception):
    pass

def parse_env_by_hand(path: str, required_keys: list[str]) -> dict[str, str]:
```

```

env_vars: dict[str, str] = {}
for line in open(path):
    line = line.strip()
    if not line or line.startswith("#") or "=" not in line:
        continue
    key, value = line.split("=", 1)
    value = value.strip().strip('\'').strip('"')
    env_vars[key] = value

missing_keys = []
for key in required_keys:
    if key not in env_vars:
        missing_keys.append(key)

if missing_keys:
    raise MissingEnvKeyError(f"Missing required key(s) in {path}: {missing_keys}")

return env_vars

# succeeds: both keys are present
values = parse_env_by_hand(".env", required_keys=["OPENAI_API_KEY", "LOG_LEVEL"])
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])

# fails on purpose: DATABASE_URL isn't in the file
try:
    parse_env_by_hand(".env", required_keys=["OPENAI_API_KEY", "DATABASE_URL"])
except MissingEnvKeyError as e:
    print(f"Startup failed: {e}")

```

Expected output:

```

sk-test123
DEBUG
Startup failed: Missing required key(s) in .env: ['DATABASE_URL']

```

Approach 3 — skip writing this at all: `python-dotenv`

```

# practice/env_config_practice.py – Intermediate section
import os
from dotenv import load_dotenv

load_dotenv()

```

```
api_key = os.getenv("OPENAI_API_KEY")
log_level = os.getenv("LOG_LEVEL", "INFO")

print(api_key)
print(log_level)
```

Expected output:

```
sk-test123
DEBUG
```

`load_dotenv()` reads `.env` and copies its values into `os.environ` for you — it already strips the surrounding quotes and already skips comment lines, for free. `os.getenv("LOG_LEVEL", "INFO")` also shows the standard way to supply a default for an optional key, instead of writing your own missing-key check for keys that aren't actually required.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate's parser only works correctly on a perfectly tidy file — no comments, no quotes. Approach 1 fixes that, so it survives a real `.env` file's messiness. Approach 2 adds a second, separate concern on top of Approach 1: after parsing, it checks the *result* is actually complete, and fails loudly with every missing key named at once if it isn't — this is the same "fail at startup, not later" idea Doc01's Build Task uses for `require_env()`, just checking a whole list of keys in one pass instead of one key at a time. Approach 3 is the odd one out: it doesn't write any parsing logic at all — it hands the entire job to a well-tested library that already does what Approaches 1 and 2 do (and more, like supporting multi-line values), in 2 lines.

Which one should you actually write? In real code, Approach 3 (`python-dotenv`) — always. The point of writing Approaches 1 and 2 by hand in this exercise wasn't to replace `python-dotenv`, it was to understand what it's doing under the hood, so it stops feeling like a black box and starts feeling like "oh, that's just a loop with a dictionary." The Intermediate versions are what you'd write if you genuinely couldn't use any library (rare). Advanced Approach 2's required-key check is worth keeping even alongside `python-dotenv` — pair `load_dotenv()` with your own small check (or `require_env()` from Doc01's Build Task) that confirms every key your program actually needs is present, since `python-dotenv` itself will happily load a `.env` file that's missing something important without complaint.